# AI SOFTWARE TESTER

## PROJECT REPORT

*A Local AI Assisted Platform for Requirement Analysis, Test Generation, Execution and Defect Reporting*

**Mini Project Report**

2026

# ABSTRACT

AI Software Tester is a locally operated quality assurance platform that connects natural language requirements with generated test cases, controlled automation, execution evidence, failure analysis and defect records. It combines a Streamlit interface, Python services, a SQLite database, an Ollama-hosted language model, Playwright browser automation and an HTTP API test engine. Its central design principle is that model output is a proposal: structured response parsing and an explicit action allow-list stand between generated instructions and browser execution. Testers can review generated artifacts, run tests against a configured target, retain outcomes and screenshots, and use dashboard and report views to inspect coverage and failures. The repository contains a Flask demonstration target with four deliberately documented defects, and pytest suites for AI response parsing, database operations, action validation and metrics. This report follows the supplied project outline and expands each section with system context, design rationale, data model, operational flow, implementation detail, verification scope, limitations and future work. Local-first execution can avoid routine external inference requests after software and model assets are acquired; it does not prove that every deployment is secure or that AI-generated tests are complete. Source reconciliation is needed: the report and README describe Qwen2.5-Coder 7B while current config.py defaults to qwen2.5-coder:3b, and the source contains 47 test functions while the supplied report says 46 tests passed. Therefore model selection and live test results should be checked before formal release. The project demonstrates a practical, reviewable workflow for AI-assisted QA while leaving test acceptance, environment configuration and defect disposition under human control.

## Report Scope and Reading Guide

The report describes the project as represented by checked-in code and accompanying materials. Chapters 1 and 2 explain the problem domain and system objectives. Chapter 3 records the development environment and requirements. Chapter 4 describes modules, input and output design, database structure and information flow. Chapter 5 presents testing and implementation evidence. Chapter 6 summarizes the outcome and staged enhancements; Chapter 7 lists technical references. Appendices provide a screen inventory and export guidance. The report distinguishes documented capabilities from recommendations and identifies discrepancies where the code and narrative differ.

# CONTENTS


- ABSTRACT
- 1. INTRODUCTION
- 1.1 Salient features of the system
- 2. SYSTEM STUDY AND ANALYSIS
- 2.1 Problem statement
- 2.2 Existing system
- 2.2.1 Drawbacks
- 2.3 Proposed system
- 2.3.1 Advantages
- 2.4 Feasibility Analysis
- 3. DEVELOPMENT ENVIRONMENT
- 3.1 Hardware Requirement
- 3.2 Software Requirement
- 3.2.1 About Python
- 3.2.2 About SQLite
- 4. SYSTEM DESIGN
- 4.1 Module Description
- 4.2 Input Design
- 4.3 Output Design
- 4.4 Database Design [Table design]
- 4.5 Data Flow Diagram / System Flow Diagram / Network Flow Diagram
- 5. SYSTEM TESTING AND IMPLEMENTATION
- 5.1 Unit Testing
- 5.2 Integration Testing
- 5.3 Validation Testing
- 5.4 Test cases
- 5.5 Implementation and maintenance
- 6. CONCLUSION AND FUTURE ENHANCEMENT
- 6.1 Conclusion
- 6.2 Future Enhancement
- 7. BIBLIOGRAPHY AND REFERENCES
- APPENDICES
- A. Output Screens
- B. Reports [optional]


Page numbers are generated by Word when opened. The numbered headings use native Word styles for navigation.

# 1. INTRODUCTION

Quality assurance starts before a browser is opened. Teams interpret requirements, identify observable behavior, select representative data, define expected results and decide how failures can be reproduced. Automation adds another layer: test authors translate those decisions into scripts, maintain selectors and fixtures, control environments and preserve links between requirement versions and execution outcomes. When activities are split across documents and tools, the effort to keep them aligned grows with the application and with each release.

AI Software Tester brings several of these tasks into one local application. A tester creates a project and target base URL, enters requirements in natural language, requests a model-assisted analysis, generates candidate test cases, reviews them, and can request browser or API actions. The application validates browser actions before Playwright executes them. Results, timing, failure descriptions, identifiers and screenshots can be stored and surfaced through dashboards, bug management and reports.

The model is integrated as a service rather than an autonomous operator. Prompt templates request structured output for requirement analysis, test generation, action generation, failure analysis and bug drafting. A response parser extracts and validates JSON, and the AI service retries malformed output up to three times. This improves predictability but cannot ensure that an interpretation is correct. Human review remains essential for security-sensitive cases, generated data and defect severity.

The platform is implemented primarily in Python. Streamlit provides the user interface; SQLite stores project data; Ollama serves a local model; Playwright supports Chromium, Firefox and WebKit; and a requests-based engine performs API tests. A Flask sample application provides a controlled target with seeded behaviors. This lets the project demonstrate navigation, validation, display and authentication tests without depending on a third-party application.

This report expands the supplied outline without changing its major sections or named subsections. It describes the system as represented in the repository. Assertions in the prior narrative are not treated as proof of a current live run. The differing model defaults and test counts are recorded so they can be reconciled before formal use.

The project has several linked objectives. It aims to shorten the time between understanding a requirement and preparing an executable test, preserve a persistent relationship between requirement and result, and make failures easier to inspect with common evidence fields. A further objective is to keep model-suggested actions inside a narrow execution interface. These objectives should be evaluated separately: generation speed does not prove correctness, traceability does not prove coverage, and a safe action list does not prove that the test data or target is safe.

The intended primary user is a tester or developer working on a web application who can create a project, run local services and review test outcomes. Product analysts can contribute requirement descriptions, but a technical operator is needed to install the model runtime and browser binaries. The system is most useful when the application has stable pages or APIs that can be exercised repeatedly and when expected behavior is documented sufficiently for assertions.

The scope includes functional and selected non-functional checks that can be expressed through the supported browser and API actions. The current design is not a replacement for load testing, formal security assessment, accessibility auditing or exploratory testing. Nor does a generated test automatically become an approved acceptance criterion. Those activities may inform test creation but require their own tools, expertise and review.

The main workflow has four kinds of information. Intent is expressed by requirements and priorities. Verification design is represented by cases, steps, data and expected results. Execution evidence records run settings, status, timing and observed errors. Disposition records analysis and bugs. Keeping these categories distinct helps a reviewer see whether a defect arises from a mistaken requirement, weak test, automation problem or product behavior.

For a useful result, requirements should be decomposed into observable statements. “The system is secure” is too broad to assert directly; a testable statement would identify the actor, resource, authorization rule and expected denial behavior. AI analysis can suggest such questions, but it cannot supply missing business policy. The tester should return unresolved assumptions to the requirement owner instead of allowing the model to invent a rule.

The application follows a local-first operational model. Local does not mean isolated from all networks: installation downloads dependencies, the target may be remote, and the Ollama endpoint is configurable. The data file and screenshots reside on the workstation. An

organization should inventory those routes and files before classifying the system for confidential work.

The report also treats the project as a software engineering artifact. Its quality depends on the clarity of module boundaries, validation contracts, database behavior, maintainability and reproducible verification. The interface is only one part of that quality. A professional evaluation should inspect the code path from prompt construction to stored result and should reproduce representative cases against the demo application.

The intended outcome is an assistive test workbench, not autonomous acceptance. A human decides which requirement interpretation is valid, which generated scenarios belong in a suite, which target is authorized, whether a failure is a product defect and how a bug should be prioritized. This allocation of responsibility is important because language models can generate plausible but unsupported details.

### 1.1 Salient features of the system

- **Local-first operation:** The UI, SQLite database, browser runner and configured Ollama endpoint are intended to operate on a developer workstation. A model must first be obtained; target applications may still require network access.
- **Requirement analysis:** Natural-language input can be analyzed for functional points, inputs, outputs, edge conditions, security concerns and open questions. Structured results can be stored with the requirement.
- **Test generation:** Prompt instructions ask for positive, negative, boundary, validation, security, UI and API cases. Candidate cases include scenario, steps, expected result and other fields supported by the response schema.
- **Reviewable automation planning:** The model proposes Playwright actions or API definitions. Generated cases are intended for tester review before they are saved or executed.
- **Allow-listed browser actions:** The validator accepts defined actions such as navigate, click, fill, select, check, uncheck, press, wait, assertions and screenshot; required fields and non-negative wait values are checked.
- **Execution evidence:** The browser runner records outcome, time, failed step, error, URL context and browser information. It can capture a failure screenshot. The API engine records response status, duration, bounded body text and assertion outcomes.

- **AI-assisted triage:** Failure prompts request classification, confidence, explanation, evidence, investigation advice and a possible fix. A separate prompt drafts an editable bug report.
- **Traceable local records:** Projects, requirements, cases, steps, runs, results, bugs, screenshots, analysis, reports and settings are stored in linked SQLite tables.
- **Dashboard and exports:** The interface includes project metrics, visual summaries and report generation. The project description lists HTML, CSV and PDF output; each format should be confirmed in the current code branch.
- **Controlled demo target:** The Flask sample application provides four documented behaviors for demonstrating navigation, validation, UI content and authentication testing.
- **Automated checks:** Pytest modules cover JSON parsing, action validation, selected database operations, metrics and execution helpers. Static inventory finds 47 test functions; this is not a fresh test result.

The features reduce clerical distance between a requirement and executable evidence. The value is not that every generated case is automatically correct; it is that the lifecycle can be inspected in one place. Productivity should be judged by the quality and maintainability of accepted cases, not by raw generation volume.

# 2. SYSTEM STUDY AND ANALYSIS

## 2.1 Problem statement

Manual test design can be thorough, but it consumes time and depends on a reviewer's experience, context and attention. A short requirement may omit data constraints, error handling or authorization expectations. If testers interpret it differently, cases become inconsistent. When a team automates the cases, it must encode the intent again in scripts and assertions, often in another repository or tool.

This separation creates practical risks. A test may no longer match its requirement after a change. A defect record may not identify the run that exposed it. A failure may lack the URL, browser, test data or screenshot needed to reproduce it. Teams can spend time classifying infrastructure and selector failures as product defects, while real defects remain hidden among noisy results.

The project addresses the need for an integrated workflow that accepts plain-language requirements, proposes structured analyses and cases, converts selected cases into a limited set of actions, runs those actions, stores evidence and assists with interpretation. The design must preserve the link between source requirement and result while controlling the transition from probabilistic model output to real browser actions.

The problem is broader than generating test text. A useful system must handle input quality, response structure, action validation, execution errors, persistence, traceability, review and reporting. If the model is unavailable or returns invalid content, the interface should show the error rather than silently substituting an unrelated model or representing a test as successful.

From a requirements-engineering perspective, the quality problem begins with incomplete, ambiguous or conflicting statements. A requirement may describe a goal without specifying the actor, trigger, precondition, observable response or failure behavior. A test tool can faithfully store ambiguous text, and an AI model can make it sound more precise, yet neither action resolves which interpretation stakeholders intended. The workflow therefore needs a visible distinction between extracted information and questions that remain open.

The project's core user need is to reduce the repetitive translation work while preserving review. A requirement owner should be able to recognize the generated scenarios as faithful; a tester should be able to inspect steps and assertions; and a developer should receive failure

context sufficient to reproduce a defect. Each role needs different details, so reports should avoid presenting a single pass-rate value as a complete quality judgment.

An effective system must also handle uncertainty at each stage. Generated test cases are hypotheses about useful coverage. Generated actions are mappings from those cases to an interface. Execution outcomes are observations from one environment and one point in time. AI failure analysis is an interpretation of selected evidence. None of these should be silently promoted to verified fact. Provenance and status labels make that uncertainty manageable.

Several constraints bound the proposed scope. The project targets a local single-user workflow; the visible schema does not include shared-user permissions. The browser engine implements a fixed set of actions; it does not support arbitrary JavaScript, drag-and-drop or file-upload operations. API assertions cover selected response properties rather than a complete contract language. These limits are useful for a prototype but should be explicit when selecting a target application.

The quality goal is therefore practical repeatability. Given the same approved case, target build, browser and data setup, the runner should perform the same defined actions, report a comparable result and retain enough context for follow-up. Exact runtime may vary, but the expected behavior and failure classification path should be stable. Repeating a run is meaningful only if the environment and state are controlled.

Functional requirements can be grouped into user-visible capabilities. Project management should create a project with a base URL. Requirement management should attach a natural-language description and priority. AI analysis should return structured findings or a visible error. Test generation should produce reviewable cases connected to the requirement. Case management should allow edits and manual additions. Execution should run selected cases and persist outcomes. Bug management should preserve reproduction detail and status. Reporting should export a selected project view.

Cross-cutting requirements concern integrity and usability. The interface should identify the active project and runtime configuration; generated content should remain editable; the user should be able to distinguish an AI proposal from an approved record; and execution should return a clear result even when a dependency fails. The database should preserve links between entities and avoid accidental duplicate identifiers. These are design targets that can guide review even when they are not each covered by an automated test.

Non-functional requirements include local responsiveness, diagnosable errors, reproducible setup, acceptable inference delay, safe target selection and traceable records. A prototype does not need enterprise throughput, but users should know when a model call or browser run is still active. Persisted data should survive restart, while temporary demo data should be removable. Reports should remain readable and should not expose secrets.

Acceptance criteria should be observable. For example, creating a requirement should make it appear in the selected project; a valid model response should produce an editable analysis; an unsupported action should be rejected before launch; a completed case should have a result tied to its run; and a bug created from a failure should retain its relation to that result. These criteria connect architecture to verifiable behavior.

Usability review should focus on reducing context switching. A tester should be able to move from a requirement to its test cases, see whether they have been executed, open the result evidence and create a linked bug. If identifiers or links are hidden, the underlying relational schema provides less practical value. Navigation should make the traceability chain visible and avoid forcing users to manually copy IDs.

Error messages should state what the user can do next. “Ollama unavailable” can suggest checking the service and endpoint. “Invalid JSON after retries” can expose a safe excerpt and allow retry or manual entry. “Browser not installed” should point to the dependency setup. “Blocked: no actions defined” should link to the case editor. Actionable errors reduce support effort and prevent misclassification.

## 2.2 Existing system

In a conventional workflow, a requirement is captured in a specification or issue tracker. A test engineer clarifies ambiguity, builds scenarios and records expected behavior. Cases may live in a spreadsheet or test-management product. Automation scripts are written separately using a browser framework. A continuous-integration system may execute the scripts, while engineers inspect logs, browser traces and screenshots.

This arrangement can work well when ownership and tooling are clear. Dedicated test-management systems may offer mature planning and access control, while browser frameworks provide direct application control. The challenge is coordination: each artifact can be sound while the relationship between requirement, case, script, run and bug remains informal. Renaming a test or changing an expected value may leave a separate automation script unchanged.

Record-and-playback utilities address some scripting effort by capturing interactions. Their output can help with simple paths, but recorded steps encode a page state and data set. They do not resolve ambiguous requirements, select boundary cases or explain whether an assertion failure indicates a product defect or an automation problem.

Cloud AI services can offer large-scale inference and hosted collaboration. They may also require sending requirement text, application details or test data to an external service. Adoption depends on data rules, vendor terms, availability and recurring cost. A local approach avoids routine external inference requests when correctly configured, while making the team responsible for model and workstation resources.

### 2.2.1 Drawbacks

Manual translation is repeated work. Every new feature or changed behavior may require revisiting a specification, enumerating scenarios, writing steps and encoding expected outcomes. Regression cycles repeat much of the effort, while time pressure often favors happy paths over less common conditions.

Coverage depends on individual judgment. Boundary values, malformed input, empty states, authorization, state transitions and accessibility are easy to miss without a shared checklist. A large case count can still leave important behaviors unverified; mapping tests to requirements and risks is more informative.

Scripts and requirements can drift. If cases live in one system and automation in another, links may depend on naming conventions or manual references. Refactoring a selector or changing an expected value may not update the source requirement record. Auditing what a run verified becomes harder.

Failure diagnosis is often manual. A failure may originate in product behavior, a stale locator, expired session, unavailable environment, missing data or transient network condition. Without consistent context and evidence, engineers reconstruct events from partial logs. This increases triage time and false bug reports.

Cloud inference adds data-handling and availability considerations. A team may be restricted from sharing specifications or data with a hosted model. Network latency and service limits can interrupt work, and per-request pricing can make repeated generation costly. These tradeoffs differ by organization.

Automated generation has its own risks. A fluent case can misunderstand a rule, invent an endpoint or assert unsupported behavior. Guessed selectors can be brittle. Without schema checks, review and execution boundaries, automation may amplify errors rather than reduce effort.

## 2.3 Proposed system

The proposed system combines requirement capture, AI-assisted interpretation, test generation, action planning, controlled execution and defect documentation in one Streamlit application. Projects store a target base URL and related requirements. The AI service sends prompt templates to Ollama and expects JSON-shaped responses. A parser extracts JSON and checks expected fields. Invalid output may be retried with a correction instruction, up to the configured limit.

After a case is reviewed, the model can propose UI actions or an API test. Browser actions pass through the same validator at generation time and again in the Playwright engine before execution. The allow-list limits action categories and checks required fields. It prevents unsupported action types from reaching the executor; it does not replace authorization, URL scope controls, selector review or careful secret handling.

The executor distinguishes API and UI cases. The API engine submits an allowed method and evaluates status, response time, selected JSON fields and headers. The browser engine launches a configured browser, creates a context and page, then processes actions in order. The first failed action returns a failure record and can trigger a screenshot. The test executor persists the result against the run and case.

Failure analysis and bug generation are assistance steps. A tester reviews classification and suggested investigation, edits a draft report and decides whether it represents a product defect. The database stores projects, requirements, test cases, runs, results and bugs in related tables, allowing project-level metrics and traceability.

The workflow can be summarized as create project, add requirement, analyze, generate cases, review and approve, generate actions, validate, execute, analyze failures, create bugs and export reports. It is feedback-aware: cases, prompts and action definitions can be corrected and rerun after diagnosis.

### 2.3.1 Advantages

- **Reduced setup friction:** Start from requirement text instead of writing every automation artifact from scratch; generated content remains editable.
- **Repeatable breadth prompts:** Prompts explicitly consider negative, boundary, validation and security behavior, subject to human confirmation.
- **One local workflow:** Requirement, case, run, analysis and defect records share a project context and identifiers.
- **Constrained execution:** Browser actions are selected from a small allow-list and checked before execution.
- **Useful evidence:** Duration, error, failed step and URL can accompany a screenshot to aid reproduction.
- **Simple deployment:** SQLite avoids a separate server, and Streamlit provides an interactive UI with little custom frontend code.
- **Local inference option:** A local model can keep prompt contents within the local environment when configured accordingly.
- **Faster triage starting point:** Structured failure analysis can suggest categories and next checks, subject to review.
- **Portable reports:** HTML and CSV support sharing and analysis; PDF can provide a stable snapshot when enabled.
- **Demonstrable behavior:** A demo target with known defects provides visible examples of functional and security failures.
- **Modular foundation:** Prompt, data, page and execution responsibilities are divided into packages.

These advantages describe the workflow, not guaranteed defect-detection superiority. Generated coverage should be compared with a risk-based test plan, and the model should be evaluated on representative requirements before it becomes a quality gate.

## 2.4 Feasibility Analysis

Technical feasibility is supported by a mature Python ecosystem and modular repository. Streamlit handles the UI; SQLite provides transactional persistence; Playwright exposes browser automation; requests supports HTTP operations; and Ollama offers a local inference endpoint. Constraints include workstation memory, model latency, browser installation, target availability and schema compatibility as the product evolves.

Hardware feasibility depends on model size and quantization. The project material specifies 16 GB RAM for Qwen2.5-Coder 7B and describes GPU acceleration as optional. Treat this as a planning estimate, not a benchmark: model build, context length and other running applications affect memory. The checked-in config currently names a 3B model, so operators must verify the active tag rather than assume 7B.

Economic feasibility is favorable for a single-user prototype because core components are open-source or locally available and local inference has no per-call cloud charge. Costs remain in hardware, electricity, setup, model storage and staff time. Local inference can be slower or less capable than hosted alternatives, and prompt curation requires effort.

Operational feasibility is helped by a browser-based interface and guided forms. Setup still requires Python packages, browser binaries and a running model. The workflow is most practical for developers and QA engineers who can configure services and diagnose target failures.

Privacy feasibility improves with local inference and persistence but depends on deployment choices. Target traffic may reach a remote system. Screenshots, API responses and prompt logs can contain sensitive information on disk. Define retention, access, masking and backup rules before using production data.

Schedule feasibility is appropriate for a mini-project because established libraries and a small local target keep the scope manageable. A successful demo does not prove compatibility with arbitrary web architectures, authentication schemes or APIs; it demonstrates bounded feasibility.

# 3. DEVELOPMENT ENVIRONMENT

The project is designed for a workstation where the UI, database, AI runtime and test runner operate as local services. The repository separates UI pages, AI integration, data access and execution engines, but they share one Python environment and a SQLite database. This makes setup compact and inspection straightforward; dependencies and configuration must remain consistent.

## 3.1 Hardware Requirement

The original project material gives a quad-core processor at 2.0 GHz or higher, 16 GB RAM, 10 GB free storage and an optional GPU. These are planning values, especially for a 7B model, rather than guaranteed performance specifications. Model weights, runtime memory, context length, browser processes, screenshots and concurrent applications affect actual use. GPU acceleration depends on runtime and drivers; CPU-only operation may be slower.

| Component | Project baseline | Engineering note |
|---|---|---|
| Processor | Quad-core CPU, 2.0 GHz+ | Suitable baseline for UI and light browser runs; inference depends on CPU generation. |
| Memory | 16 GB RAM | Planning minimum for described local model; verify simultaneous model and browser usage. |
| Storage | 10 GB free | Allow additional room for weights, Python, browser binaries, database, reports and screenshots. |
| Graphics | Optional GPU | Acceleration depends on supported drivers and Ollama configuration. |
| Network | Local / localhost | Model inference can be local after downloads; target URLs and installation may require network. |
| Display | Desktop browser | Used for Streamlit, and optionally headed Playwright runs. |

A constrained machine should start the model and app independently, observe memory use and select an appropriate model size. The current code default differs from the narrative, so runtime verification is necessary. Use a non-production target unless automation against a live service has been explicitly approved.

Environment setup should be reproducible from a clean checkout. The setup guide should name supported Python versions, install dependencies into an isolated environment, install exactly the Playwright browser binaries required, obtain the model tag and verify the Ollama endpoint. A fresh environment test catches dependencies that were installed manually but omitted from requirements.

Configuration should be validated at startup. An invalid base URL, unsupported browser name, negative timeout or missing model should produce a direct message. The application should show which values came from defaults and which came from persisted settings, because a stale database setting can survive a code update and confuse the user.

Local model performance has user-facing consequences. A prompt can take long enough that users may click a button again or assume the app has stopped. The interface should display request state, elapsed time and cancellation behavior where supported. When a request times out, it should not leave an ambiguous half-generated record.

Browser selection is a compatibility choice as well as a coverage choice. Chromium is a convenient default, but cross-browser regressions require running the same case in multiple engines and recording each result independently. A test passing in Chromium says nothing about Firefox or WebKit unless those browsers were actually run.

Test timeouts need a consistent interpretation. The Playwright engine sets a default timeout for locators, while the API engine uses a request timeout in seconds. Configuration labels should make units clear and prevent a single numeric setting from being interpreted differently across engines.

Headless execution is suitable for unattended runs, whereas headed mode can help a tester observe a failure. Both modes should use the same assertions. A headed run may be slower or affected by display state, so the selected mode belongs in the execution record.

Application lifecycle should be explicit. A run can be Pending, Running, Completed or Aborted; if a process closes unexpectedly, stale Running rows need a recovery path. On restart, the UI might identify interrupted runs and allow a user to mark them aborted without pretending that all cases completed.

Backup and restore should be tested, not assumed. A backup procedure should include a consistent database snapshot and all referenced files, record the application/schema version and confirm that a restored copy can open a project and inspect prior results. Copying an actively written database file may produce an inconsistent backup unless SQLite backup facilities or a quiescent window are used.

## 3.2 Software Requirement

Project materials identify Windows 10/11 and note Linux/macOS support; Python 3.10 or later; Streamlit; Ollama; Playwright; SQLite; Flask; pytest; and Plotly. The dependency

manifest is authoritative for packages in the checked-out version, while the README documents setup.

| Layer | Technology | Role |
|---|---|---|
| Language | Python 3.10+ | Application logic, service integration, persistence and tests. |
| UI | Streamlit | Multi-page forms, status views, charts and reports. |
| AI runtime | Ollama | Local HTTP endpoint for model prompts and generation. |
| Model | Qwen2.5-Coder family | Produces structured analyses, cases, actions and failure summaries; exact tag must be verified. |
| Browser | Playwright | Executes validated actions in Chromium, Firefox or WebKit. |
| API | requests | Sends HTTP calls and evaluates response assertions. |
| Database | SQLite | Persists entities in a local relational file. |
| Demo target | Flask | Local pages and endpoints with seeded defects. |
| Testing | pytest | Runs unit and integration checks. |
| Charts | Plotly | Displays dashboard aggregates. |

Installation follows repository instructions: create an isolated environment, install requirements and Playwright browsers, obtain and start the selected Ollama model, optionally launch Flask, and start Streamlit. Reproducible deployment should record package versions, model tag, OS, browser build and schema version.

### 3.2.1 About Python

Python is an interpreted high-level language used for automation, data processing and web applications. One language can serve Streamlit, the Ollama client, JSON validation, Playwright, HTTP requests and database operations, reducing translation overhead between layers.

The `ai` package contains prompts, a client and response parsing. `database` defines schema and CRUD helpers. `testing` contains validation and execution engines. `pages` contains Streamlit screens. Tests import modules directly, allowing many behaviors to be checked without driving the complete interface.

Flexibility makes rapid development easy but increases the importance of explicit interfaces and validation. The AI layer returns decoded dictionaries and downstream code expects particular keys. The action validator is one explicit contract. Type annotations improve readability but do not replace runtime checks.

Reliable deployments should pin dependencies, keep secrets out of source and prompts, set operation timeouts and handle unavailable services with clear messages. Logs and exports should be reviewed for sensitive data.

### 3.2.2 About SQLite

SQLite is an embedded relational database that stores data in a file without a dedicated server. This suits a local single-user prototype: setup is simple, backups are easy to explain and Python can access it through its standard library. The database path is configured relative to the repository.

Connections enable foreign keys. Projects parent requirements, cases, runs and bugs. Requirement deletion can null a case reference; project deletion cascades through related data. Results connect runs and cases; screenshots and analyses may connect to results; bugs may reference a result.

The data helper uses parameterized values for ordinary queries and commits or rolls back through a context manager. Some update helpers interpolate column names from keyword arguments, so callers must use trusted names. Future hardening could validate column names or use explicit update functions.

SQLite works well for local development and small workloads. Multi-user collaboration and high-volume parallel execution would require reconsidering connection management and storage. Backups should capture the database and referenced image/report folders together; schema migrations should be versioned.

# 4. SYSTEM DESIGN

The design connects testing intent to execution evidence through a presentation layer, AI-assisted generation layer, validation and execution layer, persistence layer and reporting views. Data moves through Python dictionaries, JSON strings and SQLite rows. The architecture favors a compact local deployment and explicit review points.

## 4.1 Module Description

The Streamlit application is organized into eleven functional pages. The main entry point configures navigation and shared project context; page modules call data helpers and AI or test services as needed.

| Module | Responsibilities |
|---|---|
| Dashboard | Project KPIs, result/severity/coverage charts and traceability view. |
| Projects | Create and manage project name, description, target base URL and status. |
| Requirements | Capture requirement fields, invoke analysis and display/store structured output. |
| AI Test Generator | Request candidate cases, support review, and generate action plans. |
| Test Cases | List, filter and edit case metadata or steps; add manual cases. |
| Test Execution | Select cases and runtime settings, execute and persist results. |
| API Testing | Configure method, endpoint, headers, body and assertions. |
| Bug Management | Create/update defects with severity, status and reproduction detail. |
| AI Failure Analysis | Classify failures and suggest root cause/investigation. |
| Reports | Build project summaries and exports. |
| Settings | Manage or display model endpoint, model, browser, mode, timeouts and screenshots. |

These are functional modules in one Python runtime, not separate processes. Long model or browser operations can block interaction unless progress and cancellation are managed. The UI should prevent duplicate submission while work is active.

The AI service exposes distinct methods for requirement analysis, test generation, UI actions, API actions, failure analysis, bug reports and recommendations. That separation makes prompt intent auditable. Configuration and persisted settings should be reconciled at startup so the displayed model matches the service actually used, and the active model should be stored with every analysis.

The execution layer determines whether a case is API-oriented, loads action JSON, invokes the matching engine and saves the result. Playwright validates again immediately before

execution. The API engine limits methods to GET, POST, PUT, PATCH and DELETE and evaluates known assertion types. This keeps page code separate from low-level execution.

The request-response path should be treated as a contract. Prompt templates define the requested fields; the response parser extracts JSON; schema validation checks required structure; and the consuming page decides whether the content is displayed, edited or saved. If a field is missing, the system should preserve the raw response for diagnosis but should not assume a default that changes the meaning of the requirement. Retrying is appropriate for formatting errors, but it cannot repair an incorrect interpretation by itself.

Temperature and context settings influence repeatability. Lower sampling variation may make similar requests more consistent, but it cannot guarantee deterministic output across model versions or runtime changes. The AI analysis table records model identity and duration, which can support comparisons if those fields are populated consistently. A future prompt-version field would make it easier to explain why an output changed after an update.

Action validation has two layers of responsibility. The parser-level schema checks that an action response has the expected shape. The action validator checks that each item has an allowed type and required fields, and the engine validates again before dispatch. This duplication is intentional defense in depth. Validation should be strict enough that unknown fields and unexpected types cannot create ambiguity in the executor.

The current action validator checks required fields and a numeric non-negative wait value. Additional semantic checks would strengthen it: cap wait durations, validate selector and value lengths, require navigation URLs to match a target allow-list, reject script-like URLs, constrain timeouts and ensure assertions use supported operators. A fixed set of action names is helpful, but parameters also influence risk.

Browser sessions should be isolated for each run. The engine creates a new browser context and page, which limits state carryover between separate runs. The target may still retain server-side data or cookies outside that context, so test setup and cleanup remain important. If user credentials are required, they should be stored outside generated prompts and substituted through controlled secret references.

The API engine creates a requests session and resolves relative endpoints using the project base URL. It handles timeout and connection errors separately, measures duration and returns a bounded response excerpt. Before using it against sensitive systems, the endpoint resolution

and redirect behavior should be tested, request bodies limited, and certificates and authentication handled according to organizational policy.

Error categories should be stable across UI and API paths. A transport failure means the runner could not complete a request; an assertion failure means a response arrived but did not match an expectation; a validation rejection means the action definition was not permitted; and a blocked result means prerequisites are absent. These categories lead to different next steps and should remain distinguishable in charts and reports.

The database stores files by path rather than as blobs. This keeps the relational file smaller and makes screenshots directly viewable, but the database alone is not a complete backup. Moving the database without its screenshots and reports can leave broken references. A project export should either package referenced files or explicitly mark them as unavailable.

Metrics have useful but limited interpretations. A high pass rate may mean the product is stable, the cases are weak, or a recent run selected only easy cases. A low automation rate may reflect intentional manual exploration, not a gap. Open bug counts are affected by triage and status discipline. Each chart should help locate records, not replace engineering judgment.

Report generation should be deterministic for a selected project and run. The export should record its filter criteria and creation time so two files can be compared meaningfully. HTML must encode user-controlled fields; CSV should preserve values without formula injection; and PDF output should use consistent page layout. These controls are particularly important because AI-generated strings are also untrusted input.

### 4.2 Input Design

Inputs are captured in forms rather than requiring testers to write automation code. Projects collect name, description and base URL. Requirements have an identifier, title, description and priority. Natural language enables domain experts to contribute, but quality improves when actors, preconditions, constraints, state changes and expected outcomes are explicit.

AI calls receive selected fields and context. Requirement analysis takes ID, title, description and priority. Test generation can receive stored analysis. Action generation uses scenario, steps, test data, expected result and base URL. Failure analysis receives expected/actual results, error, failed step, URL, browser and timestamp.

API definitions use structured method, endpoint, headers, body and assertions. This is safer than a free-form command because the engine can validate the method and interpret assertion types. Supported checks include status, response time, selected JSON fields and headers.

Validation should occur before persistence and external interaction. Required fields should be present; URLs normalized; enumerated status and priority values constrained; JSON parsed; and actions checked against their schema. Secrets should be masked in the UI, screenshots, logs and exports. Local execution does not eliminate local exposure.

Review is an input-design feature. Testers should edit scenarios, remove irrelevant cases, clarify expected results and correct selectors before approval. Provenance should indicate whether a case was AI-generated or manually authored.

### 4.3 Output Design

Outputs support different decisions. Requirement analysis helps assess testability; candidate cases support coverage planning; action plans expose intended behavior; execution results compare observation to assertion; failure analysis assists triage; bug drafts create editable records; dashboards provide project-level summaries.

A UI result should identify case, scenario, status, duration, browser, error and failed step. URL and screenshot improve reproduction. A pass means configured actions and assertions completed; it does not prove full requirement satisfaction. Blocked cases must remain distinct from product failures.

API results include method, endpoint, status, duration, a bounded response excerpt, passed checks and failed checks. Excerpts may omit useful diagnostic fields. Authorization headers, tokens and personal data must not leak to views or exports.

Dashboard metrics aggregate stored records. Pass rate is calculated from execution results; automation rate counts cases marked Automated; open bugs include Open and In Progress; critical count groups severity. Definitions should be visible: historical repeated runs can give frequently executed cases greater weight than cases run once.

Reports package project context, cases, results and bugs. HTML supports browser viewing, CSV supports analysis and PDF can provide fixed layout where enabled. Exports should include creation time, selected run, status definitions and any filters. Files may contain sensitive details and need retention controls.

## 4.4 Database Design [Table design]

The schema defines ten tables with integer primary keys. Human-readable requirement, case and bug IDs are stored as separate fields. Timestamps retain creation/update metadata. JSON text fields hold AI analyses and action plans. The linked structure allows project filtering and traces requirement to defect.

| Table | Content and role |
|---|---|
| settings | Key/value preferences, including endpoint, model, browser, headless flag, timeout and screenshot setting. |
| projects | Name, description, base URL, status and timestamps; root scope. |
| requirements | Project reference, identifier, title, description, priority, status and analysis JSON. |
| test_cases | Project/requirement links; scenario, type, data, expected result, priority, severity, automation, status and actions. |
| test_steps | Test-case link, step sequence, action and expected text. |
| test_executions | Project, run name, browser, headless flag, base URL, status and timestamps. |
| execution_results | Run/case links, result, duration, error, failed step, actual outcome, URL, browser and analysis. |
| bugs | Project and optional result reference, ID, title, severity, priority, status, environment and reproduction fields. |
| screenshots | Optional result link, file path, label and capture time. |
| ai_analysis | Optional result, type, prompt, raw output, parsed JSON, model and duration. |
| reports | Optional project/run, name, format, path and creation time. |

The DDL includes eleven tables when counting `settings`; the project narrative calls it ten interlinked tables. Static inspection shows settings plus projects, requirements, test_cases, test_steps, test_executions, execution_results, bugs, screenshots, ai_analysis and reports: eleven in total. This is another small count discrepancy to fix in formal documentation.

Relationships are generally one-to-many from project to requirements and cases; one requirement can have many cases; a case can have ordered steps and results across runs; a run groups results; a result can have screenshots or analysis; a bug can point to the result that exposed it. Run environment data is stored separately from case definition.

Foreign-key actions encode lifecycle behavior. Project deletion cascades through dependent records, useful for disposable data but destructive for audit history. Requirement deletion nulls case references; deleting a result can remove screenshots and analyses while preserving a bug with a null result. Real deployments may need archival or soft deletion. Backups should include the database and referenced file folders.

Human-readable IDs are not declared unique in the visible DDL, so application logic should guard against duplicates within a project. Indexes on foreign keys and filters can improve

scale. JSON fields should be validated on write and read. A schema migration strategy should be used as entities evolve.

Data ownership should be explicit. A project is the natural boundary for requirements, cases, runs and bugs. A result belongs to both a run and a case, which lets the same case have a history of outcomes. If a case is edited after execution, historical results should still be interpretable; retaining a case version or snapshot of the executed steps would improve auditability. The present schema links results to the current case row and does not visibly version the case definition.

Transactions matter when one user action writes several related rows. Saving a case and its steps should be atomic, so an error cannot leave a case with only half its steps. Creating a run and populating result records should preserve enough state to identify an interrupted run. The database helper commits each execute call, so operations spanning multiple helper calls may not share one transaction unless an explicit transaction boundary is added.

Identifier generation should be concurrency-safe if multiple users are added. A count-based next bug ID can collide when two records are created at the same time or when older IDs are deleted. A database uniqueness constraint combined with a retry strategy, or a monotonically increasing internal sequence, is safer. For a local single-user demonstration, the simple helper is adequate but should not be assumed safe for concurrent use.

Foreign-key enforcement is enabled per connection, which is important because SQLite requires that setting on each connection. Data integrity checks should verify no orphaned records exist after bulk import or migration. The UI should also handle deletion cascades carefully and communicate that deleting a project can remove its related evidence.

AI records can contain both raw prompts and raw model responses. These are valuable for debugging quality but can duplicate requirement text, test data or errors. A retention policy could keep parsed outputs longer than raw prompts, redact known sensitive fields before storage or allow per-project control over prompt logging.

The screenshot and report paths are persisted as text. Absolute paths can become invalid when the project moves to another machine. Relative paths rooted in a managed artifact directory, plus an export/restore operation, would improve portability. File existence should be checked before rendering a link, and deletion rules should avoid removing evidence while a result is retained.

The action allow-list supports a deliberately small interaction vocabulary. This makes the validator easier to test and review, but a new action introduces a new permission. Each extension should specify accepted fields, value types, bounds, timeout behavior, error mapping and tests. Capability should be added only when a real test need exists.

An endpoint allow-list complements action validation. Without one, a valid `navigate` action could point to an unintended host if the project base URL or model-generated URL were changed. A safe policy would resolve relative paths against the project base and reject absolute URLs outside approved schemes and hosts. Redirect destinations should be checked as well when target isolation is required.

### 4.5 Data Flow Diagram / System Flow Diagram / Network Flow Diagram

The flow begins when a tester creates a project and enters a requirement. Requirement and metadata go to the local Ollama endpoint. Output passes through extraction and schema validation before display or persistence. Test generation uses the requirement and optional analysis. The tester reviews candidates and chooses cases. Action generation turns a case into proposed browser or API definitions.

For UI cases, the action list is checked by the allow-list before storage and again by Playwright before execution. The browser opens the target and processes actions in order. API cases are sent through the HTTP engine with an allowed method; requested assertions are evaluated. The executor writes outcomes against a run and case.

A failure may be sent to the model with execution context. Classification and suggested cause remain advisory. A tester may edit and save a defect linked to the project and result. Dashboard and reports summarize stored metrics. Cases and actions can be corrected and rerun after diagnosis.

| Stage | Input | Processing | Persistent output |
|---|---|---|---|
| Define | Project and requirement | Validate fields and assign identifiers | Project and requirement rows |
| Analyze | Requirement fields | Prompt, local call, JSON/schema checks | Analysis and AI record |
| Generate | Requirement and analysis | Propose structured cases | Reviewed case and steps |
| Plan | Case, steps, base URL | Propose actions and validate shape | Action JSON |
| Execute | Cases and run settings | Playwright or requests engine | Run, result and screenshot reference |
| Triage | Failed result and evidence | Optional classification and bug drafting | Analysis and editable bug |
| Report | Project records | Aggregate metrics and export | Dashboard and report files |

The architecture can be represented as Tester → Streamlit → Python services → (Ollama generation; validator → Playwright; API engine) → SQLite and local files. The target application receives browser or HTTP traffic from the runner. Ollama traffic is intended for localhost, but endpoint settings can change this. Verify the endpoint and active model before sending project content; this design is not by itself a network security guarantee.

The trust boundary lies between generated content and execution. JSON validation limits structure, the allow-list limits browser operations, and fixed dispatch avoids evaluating model text as code. This is not a complete sandbox. Broader deployments should restrict target hosts, limit payloads, apply timeouts, isolate browser profiles, redact secrets and review filesystem access. The current navigate action accepts a URL and should be paired with host restrictions for sensitive targets.

A network-flow review should identify four paths: browser traffic to the target, API requests to the target, prompt traffic to the Ollama endpoint and report/file access on the workstation. If Ollama is bound beyond loopback or the target is remote, traffic crosses additional network boundaries. A deployment inventory should name each destination, protocol, data type and owner. This makes the phrase “runs locally” precise rather than implying that no network traffic exists.

Privacy controls should consider prompt retention. Requirement descriptions can reveal product plans; test data can resemble customer records; failure messages can contain identifiers; screenshots can expose names or account state. Avoid using production personal data in demonstrations. Use synthetic data and redact exports. If raw prompts are stored for debugging, access and retention should be limited.

Application logs should distinguish operational detail from user-facing output. Logs can record correlation ID, stage, duration and error category without copying a full request body or credential. A support workflow should let the user provide sanitized diagnostics. These choices reduce accidental exposure while retaining enough information to diagnose local setup problems.

The trust model includes the target application. Automated tests can submit forms, modify records or trigger emails. Even a valid allow-listed click can have significant side effects. Users should choose a test tenant, seeded database or sandbox and understand cleanup behavior before executing a suite. The action validator governs action type, not business impact.

Before execution, the user should confirm project, base URL, browser, headless mode and selected cases. A summary step can show whether any case sends a destructive request, uses a production URL or contains unmasked test data. Such a review screen is especially useful when AI-generated actions are approved in batches.

# 5. SYSTEM TESTING AND IMPLEMENTATION

Verification should cover individual functions, connected modules and application-level examples. The repository includes checks for parsing, validation, database helpers and metrics, plus a scripted demo workflow. The materials describe a broader demonstration in which seeded bugs are detected. This report does not claim a new test execution: historical outcomes are attributed to the supplied project narrative. A release should record environment, command, model configuration and results.

## 5.1 Unit Testing

Unit tests isolate functions with known inputs and expected outputs. `test_ai.py` covers plain, fenced, embedded, invalid and trailing-comma JSON; schema acceptance and rejection; and action validation. Tests check that required fields are enforced and unsupported operation types are refused.

Database tests use temporary SQLite files to verify settings, project, requirement, test-case, step and bug operations. They check IDs, retrieved rows, counts and step ordering. Temporary databases prevent contamination of the working project data. Cleanup should remain reliable even after assertion failures.

Metrics tests create controlled mixes of pass/fail results and verify percentage, bug counts and empty-project behavior. These checks matter because dashboard values can silently mislead if status labels or denominators change.

The executor test module mainly covers validation rather than launching a browser. It checks field definitions, rejection of dangerous action names, malformed lists and mixed valid/invalid input. These are useful fast checks but do not replace a real Playwright integration run.

Static inventory finds 47 named test functions across four pytest modules: 17 AI, 16 database, 8 executor and 6 metrics. The supplied report narrative says 46 passing tests. Resolve this by running the current suite and recording collected and passed counts. Source inventory is not evidence that all tests pass.

| Suite | Coverage | Examples |
|---|---|---|
| test_ai.py | Parsing, schema and actions | Embedded JSON, missing fields and disallowed actions. |
| test_database.py | CRUD and ordered steps | Project/requirement/case retrieval, step retention and bug IDs. |

| | | |
|---|---|---|
| test_executor.py | Action allow-list | Required fields, shell-like labels and malformed collections. |
| test_metrics.py | Aggregate behavior | 0%, 75%, 100%, bug counts, empty projects and requirements. |

The response parser is a reliability boundary because language models may wrap JSON in code fences or include explanatory text. The project tests extraction from plain, fenced and embedded output and a trailing comma case. In production, parser repair should remain conservative: removing a harmless fence is different from guessing a missing assertion or inventing a required value. If the repaired structure is uncertain, fail visibly and request review.

Retry behavior should be bounded. The service retries schema failures up to three attempts and adds a correction instruction after invalid output. A network/runtime error returns early rather than repeatedly issuing the same failed request. This reduces wasted inference, but a retry can produce different content. The UI should indicate that a later attempt generated the displayed result and preserve enough context for diagnosis.

Prompt templates are part of the application's behavior and should be version-controlled like code. Changes should be reviewed for output schema, safety language, coverage instructions and accidental inclusion of sensitive information. A prompt update should be accompanied by parser tests and representative evaluation examples so that formatting changes do not quietly break persistence.

Generated requirements analysis should separate facts stated by the user from assumptions suggested by the model. Useful fields include actors, preconditions, inputs, outputs, business rules, error behavior, security concerns, edge cases and open questions. Open questions should remain visible rather than being filled with plausible guesses. A reviewer can then return them to the requirement owner.

Generated cases should be evaluated for independence, testability and expected-result clarity. Duplicated cases inflate counts without increasing coverage. A case that says "verify the page works" is difficult to automate because it lacks an observable condition. A reviewer should remove duplication, split cases with unrelated assertions and ensure each expected result is falsifiable.

Generated browser actions rely on selectors. Stable application identifiers such as accessible roles, labels or deliberate test IDs are generally more maintainable than positional selectors or layout-dependent CSS. The current action model accepts selector strings, so prompt guidance

and reviewer inspection are important. A selector that happens to match today can become invalid after a harmless redesign.

API response-time assertions need controlled expectations. Local machine load, model activity and network conditions can cause variation. A threshold should reflect a stated service-level requirement or a broad regression guard, not an arbitrary very low value. The API engine measures elapsed request time locally; it does not separate DNS, connection, server and transfer components.

Assertion semantics should be documented. Text matching may be case-insensitive containment unless exact mode is selected; URL assertions use containment; JSON-field paths traverse nested objects; and header checks support selected operators. A tester should know these semantics because a test's label can imply stronger equality or normalization than the engine actually performs.

An execution summary should preserve both case-level and run-level status. A run may complete even when one case fails; a run can be aborted before every selected case starts; and a case can be blocked because no action plan exists. Summary logic must say whether it counts only completed results and how blocked or skipped cases affect percentages.

Demo validation should include both the defective and corrected behavior. For each seeded issue, run the case against the original demo to establish that it fails, apply a controlled fix or use a known-good build, and run again to establish that it passes. This positive/negative pair demonstrates test sensitivity and reduces the chance that a permanently failing test is mistaken for successful defect detection.

## 5.2 Integration Testing

Integration tests verify adjacent modules exchange data correctly. The main path crosses Streamlit, AI service, Ollama client, parser, database and display. A deterministic stub response can verify prompt wiring and persistence without a live model; a separate smoke check can confirm the configured endpoint and model are reachable.

Test generation should confirm requirement identifiers and analysis reach the prompt; returned cases parse; reviewer approval saves them with correct project and requirement references; and ordered steps remain intact. Malformed output, timeouts and retry exhaustion should result in a visible error and no partial case.

The action path should test both success and rejection. A valid plan should serialize and later pass the executor boundary. Unsupported types or missing fields should be rejected before browser launch. Testing this connection proves the boundary is invoked at the right point, beyond isolated validator tests.

A persistence test can create a run, execute or stub cases, store results, attach screenshots, retrieve results with case IDs, create a linked bug and verify dashboard metrics. Failure injection should cover browser startup, target connection and database errors.

Report integration tests should compare exported content to selected records and check escaping and sensitive-value handling. HTML should escape untrusted text; CSV must quote commas and newlines; PDF should be tested on the supported platform and checked visually.

### 5.3 Validation Testing

The Flask target is a controlled validation environment with login and registration pages, a dashboard and JSON endpoints. Its documented seeded defects cover navigation, form validation, rendered identity and authentication. A tester can compare behavior with expected outcomes using UI or API cases.

BUG-DEMO-001 redirects a successful login through `/home` before `/dashboard`. A test should distinguish final navigation from an intermediate redirect and define whether it checks a redirect response or the final page, avoiding false failure from a valid redirect chain.

BUG-DEMO-002 accepts mismatched registration password and confirmation values. A negative case should submit different values, assert a clear rejection and verify that no account was created. Unique test data avoids collisions across runs.

BUG-DEMO-003 displays a fixed greeting. Authenticate as a known user and assert the rendered greeting includes that identity. This checks content, not merely successful page loading.

BUG-DEMO-004 accepts any non-empty password for a demo account. A security case should submit a wrong password, verify denial and confirm protected content remains unavailable. Exercise this only against the local demonstration target.

The supplied narrative says the four bugs were detected and 46 tests passed. These are historical claims, not newly reproduced outcomes here. The checked-in tests have 47 named functions. A fresh run should record exact case IDs, environment, model and screenshots.

| Defect | Test intent | Expected observation | Category |
|---|---|---|---|
| BUG-DEMO-001 | Valid login; inspect final navigation | Confirm destination and distinguish redirect hop from wrong landing | Navigation |
| BUG-DEMO-002 | Register with mismatched passwords | Validation appears; registration does not proceed | Input validation |
| BUG-DEMO-003 | Login as named user and inspect greeting | Greeting uses authenticated name | UI/content |
| BUG-DEMO-004 | Wrong non-empty password for demo account | Authentication denied; protected content unavailable | Security |

A test plan should identify what will be exercised, what is excluded, the target build, environment, data setup, browser and any dependencies. If a test depends on an external service, the plan should state whether that dependency is live, stubbed or simulated. An execution record without this context may be difficult to compare with another run even when the case identifier is unchanged.

Test levels answer different questions. Unit checks identify defects in parsing, validation or aggregation functions. Integration checks show that data crosses module boundaries as intended. System validation exercises the running application against a controlled target. Acceptance testing confirms that stakeholders agree the behavior satisfies a business need. The current repository has meaningful unit coverage and a demo validation path; a formal acceptance process would require stakeholder-defined criteria beyond the code.

Test coverage should be measured against a model of behavior rather than only against lines of code or generated-case totals. For a requirement, reviewers can enumerate conditions and outcomes, then mark which case verifies each one. The existing traceability matrix is a foundation; future coverage views could show requirements with no approved case, cases without a requirement and cases that have never completed successfully.

Execution repeatability requires stable test data. A test that creates a user should either remove the user after the run or use a unique identifier. A test that assumes a fixed account should verify its initial state. Parallel tests must not mutate shared records in conflicting ways. The demo app’s known credentials make demonstrations convenient but should not be copied into real environments.

Assertions should be chosen to detect the intended failure while avoiding incidental implementation details. A URL assertion may be appropriate when route behavior is part of the requirement; otherwise a visible authenticated page may be a more stable outcome. Exact text assertions are useful for content requirements, while partial matching can accommodate harmless surrounding text. Every assertion should answer a question that matters to the requirement.

When a test fails, first establish whether the action ran and the target was reachable. Then inspect the failed step, URL, screenshot and logs. Compare actual behavior with the expected result and rerun only after deciding whether the environment was transient. If a failure is reproducible and contradicts an approved requirement, create a bug with minimal steps and precise data. Avoid classifying a failed locator as a product defect without inspecting the page state.

AI failure analysis should be assessed for calibration. A confidence label is a model-generated estimate, not a statistical probability unless calibrated against labeled data. The evidence and recommendation fields should be reviewed against the original execution. Teams can track disagreement between AI classification and human disposition to identify prompt weaknesses and categories that need better evidence.

The test report supplied with the project mentions 46 passing tests and four seeded defects detected. A formal validation record should include the command, date, commit, Python environment, browser, target URL, model tag and exact output. Separate automated-suite results from demo findings because they answer different questions. A test suite can pass while a particular demo scenario is not run, and a demo can expose a defect even when unit tests pass.

Negative security scenarios should use harmless, authorized inputs and a controlled target. The demo authentication bypass is intentionally seeded for this purpose. Broader security work should not be inferred from that example; the platform does not constitute a penetration-testing framework. Tests that can create, delete or alter data should be isolated and documented before execution.

Failure screenshots are useful but can contain personal information or active session data. Capture only when needed, store with restrictive permissions and apply retention. When publishing the report, replace sensitive images with redacted versions or omit them. A screenshot is evidence of one rendered state, not a complete execution trace.

### 5.4 Test cases

A useful case has a stable identifier, concise scenario, preconditions, test data, ordered steps, expected result, priority, severity and type. The schema also records automation classification and optional action JSON. These fields support execution and reporting, but scenario and expected result are the core: observed behavior must be comparable with an agreed expectation.

Design begins by mapping requirements to observable outcomes. Login should cover valid and invalid credentials, empty fields, unknown users, session behavior and protected pages where specified. Registration should cover required fields, malformed values, duplicates, password rules and confirmation mismatch. APIs should include status, response structure, headers, errors, authorization and response time when required.

Positive cases confirm intended use. Negative cases verify rejection of invalid or unauthorized use. Boundary cases probe limits. Validation cases inspect feedback. Security cases examine access control and unsafe input but must remain authorized and bounded. UI cases check visible state; API cases verify protocol contracts. A scenario may serve multiple categories, but its purpose should be explicit.

The result vocabulary includes Not Executed, Pass, Fail, Blocked and Skipped. A blocked result means prerequisites or automation are missing; skipped means an explicit decision not to run; fail means an assertion/action did not meet its condition. Treating all non-pass states as failures distorts defect counts and pass rates.

| ID | Type | Scenario | Expected result |
|---|---|---|---|
| TC-LOGIN-001 | Positive / UI | Sign in with active user and valid credentials | User reaches authenticated dashboard and sees correct identity. |
| TC-LOGIN-002 | Negative / UI | Sign in with known email and incorrect password | Authentication denied; protected page unavailable. |
| TC-LOGIN-003 | Validation | Submit login form with required fields blank | Missing values are identified; no authenticated session. |
| TC-REG-001 | Negative / validation | Submit different password and confirmation | Registration is rejected with mismatch message. |
| TC-API-001 | API / contract | Request health endpoint | Documented status and JSON health field are returned. |
| TC-API-002 | API / security | Submit invalid login credentials | Specified unauthorized response and no token. |
| TC-SEC-001 | Security | Request protected page without session | Denied or redirected as requirement specifies. |
| TC-BOUND-001 | Boundary | Submit maximum field length and one character beyond | Valid limit accepted; out-of-range value rejected. |

These are report examples, not a claim that the cases exist in the current database or have run. Adapt selectors, endpoints, setup and expectations to the target. Data should be repeatable and isolated; secrets should not be stored in plain text or exported without masking.

A representative functional test starts with an approved requirement and defines the initial state, actor, data and expected outcome. The tester creates the case, reviews its steps, generates actions and checks selectors. An execution record captures the browser and target. If an assertion fails, the tester compares screenshot and actual page with the expected

behavior, then classifies the outcome. If it is an application defect, the bug stores concise reproduction steps and links back to the failing result.

A representative API test starts with a contract statement such as “health returns status ok.” The tester chooses GET, endpoint and expected status, then adds a JSON field assertion. The engine sends the request, measures duration and returns passed/failed assertions. If the route is unreachable, the result should communicate a connection error; if the server responds with a wrong status, it should show an assertion failure. These outcomes require different corrective actions.

A generated case should be reviewed against the requirement before actions are generated. Reviewers can check whether the scenario covers a real user outcome, whether test data is safe, whether the expected result is supported and whether the steps are sufficiently specific. Cases with unresolved business assumptions should be edited or held rather than approved just because they parse successfully.

When an existing case is changed, prior execution outcomes remain relevant but may no longer describe the new case definition. A future versioning feature could record a snapshot of scenario, steps, expected result and action JSON with each execution. Until then, teams should avoid rewriting a case in a way that obscures what an earlier run actually tested.

A useful run review ends with a decision: accept the result, correct the automation, retry an environment failure, or create a defect. The dashboard can summarize the run, but the decision should be based on case-level evidence. Recording the disposition helps teams avoid counting a selector repair as a product fix or closing a bug without retesting.

### 5.5 Implementation and maintenance

The application runs as a local Streamlit process. Ollama runs separately and serves the configured model; Flask can run on its own port; SQLite is initialized from the schema; Playwright browser binaries are installed through its setup command. These independent services need clear status messages so users can identify a missing prerequisite.

Configuration is split between `config.py` constants and persisted SQLite settings. The model values currently conflict: code defaults to `qwen2.5-coder:3b`, while schema seeds `7b` and README/report describe 7B. A robust startup should make one value authoritative, display the active tag and record the exact model with each analysis. Silent substitution should be avoided.

Maintenance includes dependency updates, model compatibility checks, database backups, pruning old files, prompt review and rerunning tests after changes. New action types require coordinated updates to allow-list, field schema, executor, prompts and tests. Database changes need migrations. Export formats require escaping and confidentiality review.

Operations should distinguish model unavailability from invalid output, browser startup from product failure, and target connection from contract failure. AI classification is advisory because the model sees only supplied evidence and can misread it.

Security maintenance includes target-host restrictions, credential handling, prompt/screenshot retention, file permissions and export review. The allow-list prevents unsupported action labels, but navigation accepts a URL and should be constrained for sensitive deployments.

A release manifest should record revision, Python/dependency versions, model tag, OS, browser version, test command and exact result counts, demo revision and date. This turns “tests pass” into reproducible evidence.

# 6. CONCLUSION AND FUTURE ENHANCEMENT

## 6.1 Conclusion

AI Software Tester demonstrates an integrated approach to local AI-assisted QA. It connects requirement analysis, candidate test design, automation planning, browser/API execution, result persistence, failure interpretation, bug drafting, dashboard metrics and reporting. Established components and a compact relational model keep the project understandable and runnable on a workstation.

The safety design treats model output as structured data that passes parsing, schema checks and action validation. The model does not receive a general-purpose code execution interface through the defined browser actions. A second validation in the Playwright engine rejects malformed or unsupported actions before the browser processes them. These controls reduce one risk class; URL scope, credentials, target authorization and retention still require deployment controls.

The demo application supplies testable navigation, validation, UI-content and authentication behaviors. The data model connects requirements, cases, runs, results and bugs. Local inference and storage can help where external data transfer is undesirable, although the model and browser still need installation and management.

The repository narrative should be reconciled before formal claims: active model defaults differ, the source has 47 named test functions while narrative says 46 passed, and the schema has eleven tables including settings while narrative says ten. A fresh suite run and runtime configuration check can settle these facts. They are fixable documentation/configuration issues, but matter for reproducibility.

The project is a useful prototype for exploring how local language models can reduce repetitive QA work while keeping testers in control. Its strongest contribution is a connected lifecycle and explicit execution boundary. Further work should emphasize provenance, environment controls and repeatable evidence before broadening generated behavior.

## 6.2 Future Enhancement

### Multiple model profiles and comparison

Compare completeness, correctness and stability across approved local models. Record model/version, prompt version, options and timing for reproducibility.

### CI/CD integration

Add a command-line entry point or API for approved suites, machine-readable exit codes and artifact export. Use protected secrets and explicit target allow-lists.

### Multi-user access and roles

Add authentication, permissions, audit history and concurrency rules before shared project editing.

### Visual regression

Capture baselines and compare screenshots with configurable thresholds and masks for dynamic regions; require human review of diffs.

### OpenAPI and contract testing

Import schemas, generate candidate cases, validate responses and report contract changes without exposing credentials to prompts.

### Safe action registry

Version action definitions, validation and executor implementations. Extensions should declare permissions, target constraints and tests; arbitrary evaluation should remain excluded.

### Traceability queries

Find requirements without approved cases, unexecuted cases, recurring failures, stale tests and unresolved severe bugs. Link aggregates to records.

### Observability

Store structured logs, traces, network summaries and model latency with configurable retention and correlation IDs.

### Migration and archival

Version schemas, add archival and safe project export/import, and include referenced files when packaging.

### Resilience and scale

Move long operations to workers with cancellation and queue limits; consider server persistence only if concurrent workload warrants it.

### Accessibility

Audit keyboard access, labels, contrast and status communication. Distinguish product failures from setup errors.

### Evaluation dataset

Measure coverage, unsupported assumptions, valid actions and human edits on curated requirements across model versions.

Prioritize by audience. A single-user prototype should first reconcile configuration and produce repeatable evidence. A team service also needs authentication, audit history, concurrency control and a shared persistence layer.

An implementation roadmap should sequence improvements by risk and dependency. First, reconcile the model setting and display the exact active model. Next, record fresh test results and align the documented suite count. Then add target-host restrictions and improve error categories. These steps strengthen trust in existing behavior before the project adds team collaboration or more action types.

The second stage can improve evidence quality. Persist prompt version, model tag and case definition at execution time; provide export manifests; add run recovery for interrupted processes; and verify backups. These changes make historical results interpretable and lower the chance that a team will compare runs produced under different settings as if they were equivalent.

A later stage can introduce asynchronous execution and CI support. A command-line runner should accept a project or exported suite, selected browser and environment variables, and should emit a stable machine-readable summary. It should return a failing process code when required assertions fail, but distinguish configuration errors from product failures. CI integration should preserve screenshots and logs as build artifacts.

Multi-user features require broader architectural work than adding a login form. The system needs a shared service boundary, authorization checks on every project and file, audit events, concurrent identifier generation, conflict handling and database migration. The storage layer

may need a server database. User roles should be designed around actual responsibilities and reviewed for data isolation.

Model evaluation should compare outputs on a fixed requirement set. Reviewers can label expected scenarios and measure useful coverage, unsupported assumptions, duplicate rate, parse success and action validity. Human edit distance can show how much correction is required. These measures should be reported alongside latency and resource use, and should not be collapsed into one score without a clear rationale.

An organization adopting the platform should begin with a low-risk internal application and synthetic data. The team can inspect generated cases, estimate review effort, measure failure triage quality and note environment setup costs. Only after repeatability and access controls are established should the platform run against higher-impact systems.

Project success can be judged by whether testers spend less time on repetitive translation while retaining or improving requirement coverage and failure reproducibility. A before-and-after comparison should use comparable requirements and include human review time, duplicated cases, defect signal quality and maintenance work. Faster generation alone may shift effort into correction rather than eliminate it.

The report’s recommendations are intentionally staged. The prototype already demonstrates a useful local workflow. Enhancements should preserve simplicity where it is valuable, but should not claim enterprise scale until concurrency, permissions, migration, recovery and audit behavior are implemented and verified.

## 7. BIBLIOGRAPHY AND REFERENCES

These sources document project technologies and software engineering practice. Online material changes; consult versions that match installed packages.


1. Ollama. “Ollama Documentation.” https://ollama.com/ (accessed 6 October 2026).
2. Qwen Team. “Qwen2.5-Coder.” https://github.com/QwenLM/Qwen2.5-Coder (accessed 6 October 2026).
3. Microsoft. “Playwright Documentation.” https://playwright.dev/ (accessed 6 October 2026).
4. Streamlit, Inc. “Streamlit Documentation.” https://docs.streamlit.io/ (accessed 6 October 2026).
5. SQLite Consortium. “SQLite Documentation.” https://www.sqlite.org/docs.html (accessed 6 October 2026).
6. Pallets Projects. “Flask Documentation.” https://flask.palletsprojects.com/ (accessed 6 October 2026).
7. pytest Development Team. “pytest Documentation.” https://docs.pytest.org/ (accessed 6 October 2026).
8. Plotly Technologies Inc. “Plotly Python Graphing Library.” https://plotly.com/python/ (accessed 6 October 2026).
9. Python Software Foundation. “Python 3 Documentation.” https://docs.python.org/3/ (accessed 6 October 2026).
10. Sommerville, Ian. Software Engineering. 10th ed. Pearson, 2015.
11. Pressman, Roger S., and Bruce R. Maxim. Software Engineering: A Practitioner’s Approach. 8th ed. McGraw-Hill Education, 2014.
12. Project source and documentation: README, config.py, database/schema.py, database/database.py, ai/, testing/, pages/, demo_app/ and tests/.

# APPENDICES

## A. Output Screens

This appendix inventories the eleven views and evidence to capture from a live run. The workspace contains application source but no standalone captures in the screenshots directory. The entries describe evidence to collect rather than fabricated screenshots. Insert live UI captures when preparing an institution-specific final copy.

| Screen | Evidence | Purpose |
|---|---|---|
| Dashboard | KPI cards, result/severity/coverage charts and traceability view | Shows stored records summarized. |
| Projects | Creation form, base URL and project list | Shows scope and target configuration. |
| Requirements | Requirement editor and saved analysis | Shows natural input and structured response. |
| AI Test Generator | Cases before and after reviewer approval | Shows human checkpoint. |
| Test Cases | Filtered list, case detail and step editor | Shows stored metadata. |
| Test Execution | Browser/mode, progress and result table | Shows run settings and outcomes. |
| API Testing | Request builder and status/assertions | Shows request and response checks. |
| Bug Management | Defect fields and linked execution | Shows triage record. |
| AI Failure Analysis | Classification and recommendation | Shows advisory result. |
| Settings | Endpoint, model and browser settings | Confirms runtime configuration. |
| Demo application | Login/register/dashboard and a seeded defect | Shows local validation behavior. |

Each capture should show application context, project and relevant case identifier. Mask credentials, personal data, private URLs and tokens. Link failure images to result records and avoid unrelated desktop content.

For the Dashboard capture, select a demonstration project with at least one requirement, several cases, one completed run and a linked bug. Confirm that the cards and charts agree with the visible records. A screenshot with an empty database can still demonstrate layout, but should be captioned as an empty-state view rather than evidence of execution.

For the Requirements and Generator captures, use a short non-sensitive demonstration requirement. Show the original text, the analysis output, and the candidate cases before approval. If a rejection or edit action is available, show one reviewed change so the reader can see that AI output is not automatically treated as authoritative.

For the execution evidence, include the selected browser and mode, the run status and one result detail. If the demo contains a seeded defect, show the exact test ID, expected behavior, actual observation and screenshot reference. Do not crop away the project or run context needed to interpret the result.

For the API page, use the local health endpoint or another harmless demo route. Capture method, endpoint, assertions and returned status. Remove authorization values and avoid exposing full response bodies when they contain credentials or personal data.

For Bug Management and AI Failure Analysis, use a defect from the demonstration target and show the relation to the failed result. The AI classification should be captioned as a suggestion. If the human triage changes its classification or severity, explain that the saved bug reflects the reviewed decision.

For Settings, capture the model tag actually loaded by the service, not merely a default shown in an uninitialized form. The current source mismatch makes this screen especially important. Include the browser and timeout only when they are relevant to the demonstrated run.

Captions should include figure number, screen name, environment, project and brief observation. A sequence of related images can show before/after state, but each image should remain legible at normal page zoom. Store original captures separately from compressed copies used in the report.

Appendix screenshots are supporting evidence rather than decoration. Each should support a claim in the main body, such as the review gate, action validation, saved result or traceability link. If a view cannot be reproduced, document the reason outside the final report and do not fabricate a replacement image.

### B. Reports [optional]

Use the Reports page to demonstrate project records outside the interactive dashboard. Generate examples from a named demonstration project, not production data. Confirm which formats are enabled in the current branch before finalizing this appendix.

| Export | Suggested content | Review points |
|---|---|---|
| HTML | Project summary, requirements, selected cases, latest results and bugs | Escaping, readability, links and print behavior. |
| CSV | One row per selected result or test with stable identifiers | Quoting, newline handling, column names and empty values. |
| PDF | Fixed-layout stakeholder summary | Page breaks, table layout, font rendering and source run. |

An example should identify project, execution, generation time, status definitions and filters. It should not imply that one run proves overall quality. Clarify whether it shows current cases, latest outcomes or all history. Exports may contain sensitive data and require retention controls.

Illustrative data dictionary (not a reproduction of an existing export):

| Field | Meaning | Example |
|---|---|---|
| project_name | Project scope | Demo Testing Project |
| requirement_id | Requirement linked to a case | REQ-LOGIN |
| test_case_id | Stable case identifier | TC-LOGIN-001 |
| test_type | Functional/API/security category | Functional |
| result | Outcome for one run | Pass / Fail / Blocked / Skipped |
| duration_ms | Elapsed milliseconds | 842 |
| bug_id | Linked defect if created | BUG-001 |
| executed_at | Result timestamp | Date/time value |